Practical 3

Part A:

Aim:

1. Distinct points (Point.java + Driver.java with main)

(a) Create class Point with private int x, y and constructor.

(b) Override toString() to return "(x, y)".

(c) Override equals(Object) to be true when both x and y match, and hashCode() to agree (e.g. Objects.hash(x, y)).

(d) In Driver.main: build a Point[] with some repeated coordinates.

(e) Count distinct points: for each point, use equals() to check whether it already appeared earlie r; count those that did not.

(f) Print "Distinct: N".

Expected output: 5 points with 2 repeats → "Distinct: 3".

Point.java:

Code:

```
import java.util.Objects;

public class Point {

    private int x;
    private int y;

    // Constructor
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    // toString()
    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }

    // equals()
    @Override
    public boolean equals(Object obj) {

        if (this == obj)
            return true;

        if (obj == null || getClass() != obj.getClass())
            return false;

        Point p = (Point) obj;

        return x == p.x && y == p.y;
    }
```

```
    // hashCode()
    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```

Driver.java:

Code:

```
public class Driver {

    public static void main(String[] args) {

        Point[] points = {
                new Point(2, 3),
                new Point(4, 5),
                new Point(2, 3),
                new Point(6, 7),
                new Point(4, 5)
        };

        int distinct = 0;

        for (int i = 0; i < points.length; i++) {

            boolean alreadyPresent = false;

            for (int j = 0; j < i; j++) {

                if (points[i].equals(points[j])) {
                    alreadyPresent = true;
                    break;
                }
            }

            if (!alreadyPresent) {
                distinct++;
            }
        }

        System.out.println("Points:");
```

```
        for (Point p : points) {
            System.out.println(p);
        }

        System.out.println("\nDistinct: " + distinct);
    }
}
```

Output:



Aim:

2. Duplicate card check (Card.java + Driver.java)

(a) Create class Card with private String rank, suit and a constructor. (b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts"). (c) Override equals()/hashCode() by rank + suit.

(d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals() to check it against the earlier cards. (e) Print "Duplicate found: <card>" the first time a repeat appears.

Expected output: a second "Ace of Spades" → "Duplicate found: Ace of Spades".

Card.java

Code:

```
import java.util.Objects;

public class Card {

    private String rank;
    private String suit;

    // Constructor
    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    // toString()
    @Override
    public String toString() {
        return rank + " of " + suit;
    }

    // equals()
    @Override
    public boolean equals(Object obj) {

        if (this == obj)
            return true;

        if (obj == null || getClass() != obj.getClass())
            return false;

        Card other = (Card) obj;

        return rank.equals(other.rank) &&
suit.equals(other.suit);
```

```
    }

    // hashCode()
    @Override
    public int hashCode() {
        return Objects.hash(rank, suit);
    }
}
```

Driver.java **Code:**

```
public class Driver1{

    public static void main(String[] args) {

        Card[] cards = new Card[6];

        cards[0] = new Card("Ace", "Spades");
        cards[1] = new Card("King", "Hearts");
        cards[2] = new Card("Queen", "Diamonds");
        cards[3] = new Card("Jack", "Clubs");
        cards[4] = new Card("Ace", "Spades");     // Duplicate
        cards[5] = new Card("Ten", "Hearts");

        boolean duplicateFound = false;

        for (int i = 0; i < cards.length; i++) {

            for (int j = 0; j < i; j++) {

                if (cards[i].equals(cards[j])) {

                    System.out.println("Duplicate found: " +
cards[i]);
                    duplicateFound = true;
                    break;
                }
            }

            if (duplicateFound) {
                break;
            }
        }
    }
```

```
}
```

Output:


PROBLEMS 3
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code - Part A
PS G:\OOP> cd "g:\OOP\Practical 3\Part A\" ; if ($?) { javac Driver1.java } ; if ($?) { java Driver1 }
Duplicate found: Ace of Spades

3. (For Advanced Learners – home) Fractions (Fraction.java + Driver.java)
(a) Create class Fraction with private int num, den.
(b) In the constructor, reduce to lowest terms: compute g = gcd(num, den) and divide both by g.
(c) Override toString() to return num + "/" + den.
(d) Override equals()/hashCode() using the reduced num and den.
(e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they are all equal and print the same.
Expected output: new Fraction(2,4) prints "1/2" and equals new Fraction(1,2).

Code:

Fraction.java

```
import java.util.Objects;

public class Fraction {

    private int num;
    private int den;

    // Constructor
    public Fraction(int num, int den) {

        if (den == 0) {
            throw new IllegalArgumentException("Denominator
cannot be zero.");
        }

        // Keep denominator positive
        if (den < 0) {
            num = -num;
            den = -den;
        }

        // Reduce fraction
        int g = gcd(Math.abs(num), den);

        this.num = num / g;
        this.den = den / g;
    }

    // Calculate GCD using Euclidean algorithm
    private static int gcd(int a, int b) {

        while (b != 0) {
```

```
            int temp = a % b;
            a = b;
            b = temp;
        }

        return a;
    }

    // Convert fraction to string
    @Override
    public String toString() {
        return num + "/" + den;
    }

    // Compare reduced numerator and denominator
    @Override
    public boolean equals(Object obj) {

        if (this == obj) {
            return true;
        }

        if (obj == null || getClass() != obj.getClass()) {
            return false;
        }

        Fraction other = (Fraction) obj;

        return num == other.num && den == other.den;
    }

    // Generate hash code using reduced values
    @Override
    public int hashCode() {
        return Objects.hash(num, den);
```

```
    }
}
```

Driver.java

```
public class Driver2 {

    public static void main(String[] args) {

        Fraction f1 = new Fraction(1, 2);
        Fraction f2 = new Fraction(2, 4);
        Fraction f3 = new Fraction(3, 6);
        Fraction f4 = new Fraction(4, 8);

        System.out.println("f1 = " + f1);
        System.out.println("f2 = " + f2);
        System.out.println("f3 = " + f3);
        System.out.println("f4 = " + f4);

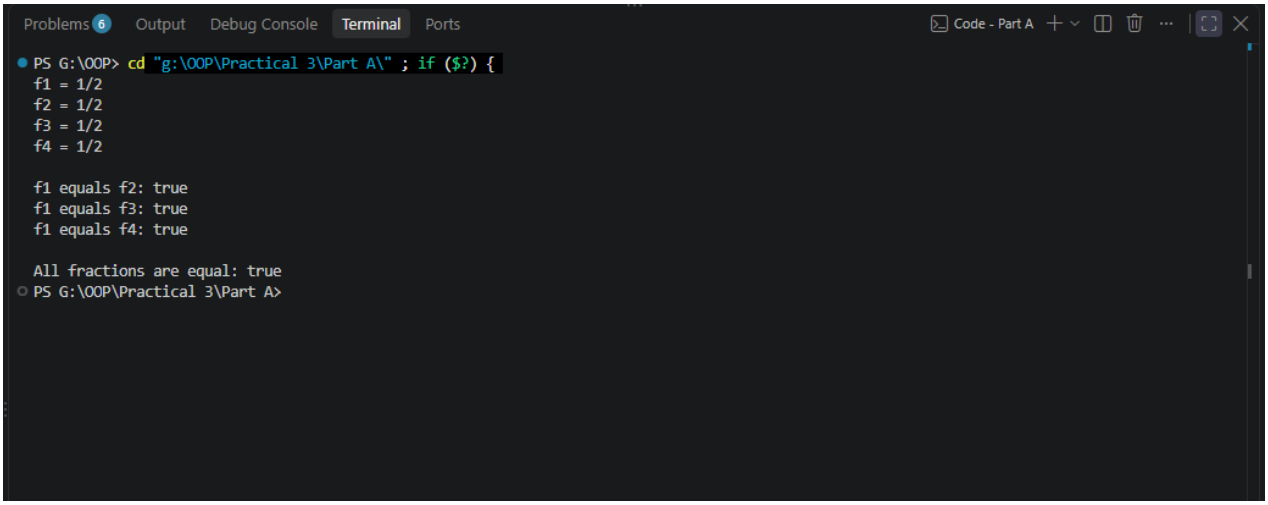
        System.out.println();

        System.out.println("f1 equals f2: " + f1.equals(f2));
        System.out.println("f1 equals f3: " + f1.equals(f3));
        System.out.println("f1 equals f4: " + f1.equals(f4));

        System.out.println();

        System.out.println("All fractions are equal: "
                + (f1.equals(f2) && f2.equals(f3) &&
f3.equals(f4)));
    }
}
```

Output


Problems 6
Output
Debug Console
Terminal
Ports
Code - Part A
PS G:\OOP> cd "g:\OOP\Practical 3\Part A\" ; if ($?) {
f1 = 1/2
f2 = 1/2
f3 = 1/2
f4 = 1/2
f1 equals f2: true
f1 equals f3: true
f1 equals f4: true
All fractions are equal: true
PS G:\OOP\Practical 3\Part A>

Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. Why must equals() and hashCode() be overridden together?

Ans.

`equals()` determines whether **two objects are logically equal**, while `hashCode()` produces a hash value used by collections such as `HashSet` and `HashMap`.

Java's contract says:

> If two objects are equal according to `equals()`, they **must have the same `hashCode()`**.

For example, in your `Account` class:

```
@Override
public boolean equals(Object o) {
    Account other = (Account) o;
    return accountNumber.equals(other.accountNumber);
}

@Override
public int hashCode() {
    return Objects.hash(accountNumber);
}
```

If two accounts have the same account number:

```
Account A → AC0001
Account B → AC0001
```

then:

```
A.equals(B)       → true
A.hashCode()      → same value
B.hashCode()      → same value
```

If you override only `equals()` but not `hashCode()`, hash-based collections may treat logically equal objects as different.

2. What does the default Object.toString() return, and why override it?

Ans.

Every Java class ultimately inherits `toString()` from the `Object` class if it doesn't provide its own version.

The default result is generally in this form:
`ClassName@hexadecimalHashCode`

For example:
```
Account account = new Account("Rahul", 5000);

System.out.println(account);
```

Without overriding `toString()`, you might get something like:
`Account@5acf9800`

This isn't useful to a user because it doesn't show the account's meaningful information.

So we override it:
```
@Override
public String toString() {
    return "Account Number : " + accountNumber
            + ", Owner : " + ownerName
            + ", Balance : Rs." + balance;
}
```

Now:
```
System.out.println(account);
```

produces something meaningful:
```
Account Number : AC0001, Owner : Rahul, Balance :
Rs.5000
```

3. What is the difference between a static nested class and an inner class?

Ans.

Static Nested Class	Inner Class
Declared with `static`	No `static`
Does not require outer object	Requires outer object
Not associated with a particular outer instance	Associated with an outer instance
Cannot directly access outer instance fields	Can directly access outer instance fields
Example: `Customer.Address`	Example: `customer.new Address()`